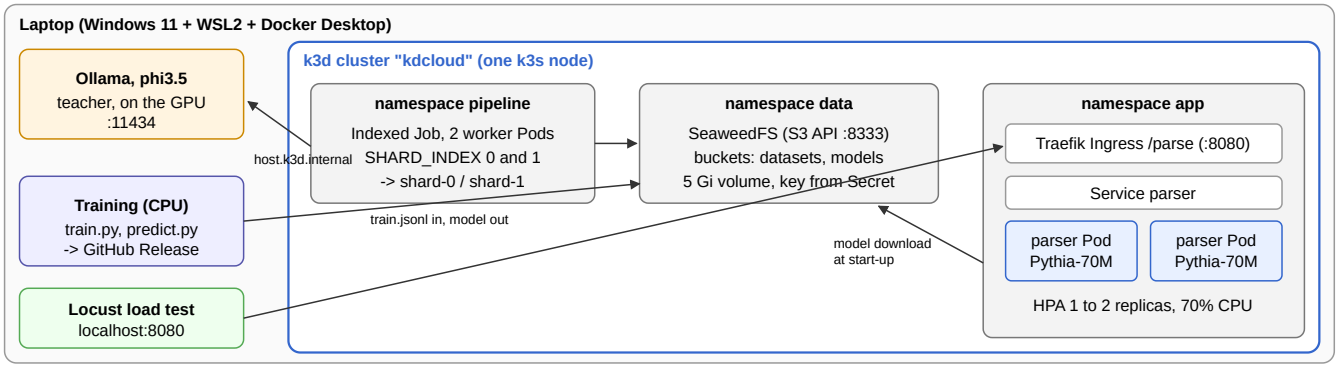
Knowledge Distillation and Deployment of a Command Parser on a Local Cloud

CS3081 Cloud Computing, mini project. Swapnil Yadav, roll number 230103015.

Repository: https://github.com/SWAPI03/cs3081-kd-parser (assessed commit: tag `final`).

Measured vs assumed. Every number in **bold** was measured on my laptop (Intel i7-14700HX, 28 threads, 16 GB RAM, RTX 4050, Windows 11 + WSL2 Ubuntu 24.04 + Docker Desktop 4.98) and comes from the file in `results/` named next to it. Assumptions are marked *Assumption*.



Architecture. Teacher and training run on the laptop; storage, labelling workers and the service run in the cluster.

1. Object storage and parallel teacher labelling

Setup. `cluster/k3d.yaml` creates a single-node k3s cluster (k3s v1.31.4): port 8080 goes to Traefik, port 8333 to the SeaweedFS S3 API (NodePort 30333). SeaweedFS 3.85 runs in namespace `data` as one Pod (`weed server -s3`) on a 5 Gi local-path volume. The S3 key is generated into the git-ignored `.env` and reaches Pods only through Secrets. `scripts/storage_init.py` creates the buckets `datasets` and `models` and uploads `commands_pool.jsonl`.

Teacher. phi3.5 served by Ollama on the laptop GPU, temperature 0, seed 42, prompt in `prompts/teacher_prompt.txt`. *Disclosed extra settings:* `format: "json"` (Ollama only lets the model emit syntactically valid JSON; the model still chooses keys, intent and slot values) and `num_predict: 96` (a length cap; a correct reply is about 30 tokens). The same function (`worker/teacher.py`) is used for labelling and for the teacher's validation score. I tuned the prompt on the first 20 validation commands: full frame went from **30% to 40%** after adding rules for vague places, time zones, parts of the day and relative times. Tuning on validation data is a small leak, so the teacher's validation score may be slightly optimistic.

Sharding. Two worker Pods run as one Indexed Job in namespace `pipeline` (`k8s/pipeline/label-job.yaml`). Kubernetes gives each Pod a completion index (0 or 1), passed to the worker as `SHARD_INDEX`; `SHARD_COUNT` is 2. A worker takes the commands whose line number modulo `SHARD_COUNT` equals `SHARD_INDEX`, so the shards are disjoint, together cover every line, and are equal in size (1,500 each). The workers never coordinate: each writes its own object (`datasets/teacher/shard-0.jsonl`, `shard-1.jsonl`) and uploads it every 25 commands. A restarted Pod reads its object back and continues after the last saved id, so a crash costs at most 25 commands and creates no duplicates. Both workers call the same Ollama server, so two workers give less than twice the speed of one; the design itself scales out unchanged with more teacher servers.

Results. The Job completed in **26 min** with **2/2** completions (`results/task1_workers.txt`), at **61 to 69 commands/min per worker** in normal operation (worker logs `results/task1_teacher-label-*.log`). At about 750 of 1,500 commands per shard, Ollama on the laptop stopped answering (HTTP 500, then connection refused). The Job replaced the failed Pods, and the new Pods resumed from their checkpoints (**2 Succeeded and 3 Failed Pods** listed in `results/task1_workers.txt`). Coverage check in `clean.py`: **3,000 ids, 0 missing, 0 repeated** (`results/clean_counts.json`).

Input	(a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept
3,000	**1**	**0**	**13**	**116**	**50**	**2,820**

Duplicates are removed first, so the copy kept is always the smallest id; each record is counted under one reason only. Rule (d) means a slot value must appear in the command as whole words after lower-casing.
`datasets/v1/train.sha256` = `011ceb9f...fedf0061`; `clean.py` is reproducible because it sorts by id, uses fixed JSON serialisation and `\n` line endings.

2. Training the student (sequence-level distillation)

Setup. `make train` reads only `training/config.yaml`: seed 42, learning rate 2e-4, batch size 16, 5 epochs, maximum sequence length 128 (the longest example is **65** tokens according to the `make train` output, so nothing is cut), linear warm-up over 5% of steps, and the library versions. Each example is `Command: <command>\nJSON: <teacher reply><end-of-text>`. The loss covers only the reply and the end-of-text token, so the student learns where to stop; this gave **100%** valid JSON. The reply is the teacher's parsed JSON written back with a fixed key order and spacing (same content, one consistent format; the raw reply is kept unchanged in the shards and in `train.jsonl`). The first 1,500 cleaned records are used. Training is a plain PyTorch loop on the CPU with Hugging Face Transformers for model and tokenizer; v2 uses seed 7. Models: `s3://models/v1/`, `s3://models/v2/`, and the GitHub Release `models` (`model-v1.tar.gz`, sha256 `7269c59f...d2675b`, equal to `models/v1/model.sha256`).

Loss per epoch (`training/loss.csv`, `training/loss_v2.csv`):

Epoch	1	2	3	4	5
v1	0.451	0.182	0.111	0.062	0.026
v2	0.453	0.197	0.121	0.059	0.026

Training v2 took **879 s** on the CPU (`make train-v2` output); v1 used the same recipe. Inference: **0.23 s** per command (`make eval` output).

Validation scores (100 commands, `tools/score.py`, files `results/teacher_validation.json`, `results/score_*.json`):

Model	JSON valid	Intent	Date	Time	Place	Full frame
Teacher phi3.5 (3.8B)	**99**	**88**	**82**	**87**	**94**	**58**
Untrained Pythia-70M	**0**	**0**	**0**	**0**	**0**	**0**
Student v1 (70M)	**100**	**84**	**81**	**90**	**85**	**51**
Student v2 (70M)	**100**	**84**	**81**	**90**	**86**	**48**

The untrained model never produces the format, so every metric is 0. After distillation, v1 reaches **88%** of the teacher's full frame with about 1/50 of the parameters. It even beats the teacher on time accuracy, because it learned the clean, consistent targets rather than the teacher's occasional rule breaks. Its main losses are intent (84 vs 88) and place (85 vs 94).

Sequence-level vs logit-based distillation. (a) *Sequence-level* distillation trains the student on the teacher's **generated output**: the teacher's reply is treated as the correct answer, and the student is trained with ordinary cross-entropy to produce the same tokens. Only text is needed from the teacher. (b) *Logit-based* (word-level) distillation trains the student to match the teacher's **full probability distribution over the vocabulary at every position** (soft targets, usually a KL divergence with a temperature), so the student also learns how likely the alternatives were. (c) It is **not possible here** because Ollama returns only the generated text, never the logits, and because phi3.5 and Pythia use different tokenizers and vocabularies, so their per-token distributions do not line up. (d) *Limitation:* the student can only copy the teacher, including its mistakes. It never sees the true labels, so the teacher's 58% full frame is effectively its ceiling, and `clean.py` removes replies the teacher got visibly wrong, which also removes some of the hardest commands.

3. The inference service in a container

Setup. FastAPI on port 8000 (`service/app.py`). `POST /parse` returns only the four-key object with `X-Model-Version` and `X-Served-By` (the Pod name); `/healthz` is always 200; `/readyz` is 503 until the model is loaded. Configuration comes only from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS` (and `FAULT_INJECT`); credentials come from the Secret `s3-credentials`. At start-up a background thread downloads the model from `MODEL_URI`. Every output is checked against `schema.json` (exact keys, intent among the eight, slots string or null); otherwise the reply is 500 `{"error": "invalid_model_output"}`. Decoding is greedy and stops at end-of-text or when the JSON object closes. The image is `python:3.11.10-slim` with CPU-only torch 2.4.1, every version pinned, no weights inside; `make image` builds it.

Results (`results/task3_container.txt`): image **1.68 GB**. With `--cpus=1 --memory=1g`, **50/50** sequential requests returned HTTP 200 in **17 s**, no OOM kill, peak memory **387 MiB** of 1 GiB. The model loaded in **6.3 s**, and `/readyz` returned 503 during the download. `FAULT_INJECT=1` returns HTTP 500 with the exact body.

4. Kubernetes deployment, scaling and reliability

Setup. Namespace `app`: Deployment `parser` (2 replicas; requests 500m CPU / 512Mi, limits 1 CPU / 1Gi), Service, Traefik Ingress for `/parse`, and an HPA from 1 to 2 replicas at 70% CPU (scale-down stabilisation shortened to 60 s so the test finishes quickly). The readiness probe is `/readyz` and the liveness probe `/healthz`. The rolling update uses `maxSurge 1, maxUnavailable 0` and a 10 s `preStop` pause, so a Pod leaves the Ingress before it stops.

Results.

Test	Measured	File
`/parse` through the Ingress	**200**, both headers	`task4_deploy.txt`
No ready endpoints while loading	`ENDPOINTS` empty, all `ready=false`	`task4_coldstart.txt`
HPA	**1 -> 2** at **77%** CPU under load (peak **196%**), **2 -> 1** after the load; events `New size: 2` / `New size: 1`	`task4_hpa.txt`
Rolling update v1 -> v2 -> rollback, under load	**0 of 1,722** failed; header **v2** after the update, **v1** after `rollout undo` (4 requests each)	`task4_rollout.txt`
Recovery (delete one Pod under load)	2 replicas again after **13 s**, **0 of 1,021** failed	`task4_recovery.txt`
Cold start (delete all Pods, 3 runs)	**9, 10, 9 s** (6 Pods: 9 9 10 10 9 9), median **9 s**; logs show the download from SeaweedFS	`task4_coldstart*.txt`
Load balancing, 2 Ready replicas	**346 / 343** requests (50% / 50%)	`loadtest_headers.json`

Limitations. The cluster is one node on one laptop, so it shows Pod-level recovery, not fault tolerance across machines: if the laptop or the node fails, everything fails. Both replicas share one CPU, so scaling out adds less capacity than on separate machines. metrics-server reports CPU only for Pods that have run for a while, so the HPA briefly had no metrics after Pods started (`FailedGetResourceMetric` events) and reacts with a delay. `maxReplicas: 2` caps throughput. The single SeaweedFS Pod is a single point of failure.

5. Subnetting and network access control

Allocation from largest to smallest, contiguous, smallest prefix that fits (`network/subnets.md`, `scripts/subnets.py`, `results/task5_subnets.txt`). `nacl_sim.py` gives **6 / 6** on the sample packets (`results/task5_nacl_check.txt`).

Subnet	Hosts	Network	Prefix	Usable range	Broadcast	Usable
app	1,000	10.20.0.0	/22	10.20.0.1 to 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0	/24	10.20.4.1 to 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 to 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 to 10.20.5.158	10.20.5.159	30

NACL (`network/nacl.json`): inbound 100 deny all from 198.51.100.0/24; 200 and 210 allow TCP 80 and 443 from anywhere; 300 allow TCP 22 from 10.20.5.128/27; 400 allow TCP 1024-65535 from 10.20.0.0/22. Outbound 100 allow TCP 8000 to 10.20.0.0/22; 200 allow TCP 1024-65535 to anywhere. Everything else is denied by default.

Why reply rules and rule order matter. A NACL is stateless: it does not remember connections, so a reply is checked like any new packet. The response to a client goes out to the client's ephemeral port (1024 to 65535) and needs its own outbound allow; the app subnet's answer comes back to the load balancer's ephemeral port and needs its own inbound allow. Rules are checked from the lowest number, and the first match wins, so the deny for 198.51.100.0/24 (rule 100) must have a lower number than the HTTP/HTTPS allows (200, 210). Otherwise the allow would match first and the abusive range could reach the web ports.

6. Reproducibility, testing and CI/CD

`make up` builds and deploys everything from a fresh clone with one command: venv, cluster, SeaweedFS, buckets, data, service image, model from the GitHub Release (checksum verified), and the Deployment. `make down` deletes the cluster. GitHub Actions (`.github/workflows/ci.yml`) runs lint (pyflakes), then **16** unit tests (cleaning rules and checksum stability, service API and `FAULT_INJECT`, subnet plan, NACL), then model evaluation (downloads `model-v1.tar.gz` from the Release, verifies its checksum, runs `score.py`), then the quality gate (`--min-validity 95`), plus a gitleaks secret scan that reports nothing. **Passing run:** https://github.com/SWAPI03/cs3081-kd-parser/actions/runs/42074609367 **Deliberately failing run** (untrained Pythia-70M, stops at the quality gate): https://github.com/SWAPI03/cs3081-kd-parser/actions/runs/42074783612 AI use is described in `AI_USE.md`.

7. Performance, cost and review

Load test (the standard command, through the Ingress, 2 replicas; `results/loadtest_stats.csv`, `results/task7_loadtest_summary.txt`): **685** requests, p50 **710 ms**, p95 **1,100 ms**, **5.75** requests/s, error rate **0%**. Each Pod used about **1,000m** CPU (its full limit) and **360 MiB** memory (`results/task7_top_pods.txt`). Cold start median **9 s** (Task 4).

Cost (`scripts/cost.py`, `results/task7_cost.txt`). *Assumption:* both replicas are billed for their limits (1 vCPU, 1 GiB) for every hour they run, at the rate card's 3.50 INR per vCPU-hour and 0.45 INR per GiB-hour. 1. One replica = 1 x 3.50 + 1 x 0.45 = 3.95 INR/hour 2. Two replicas = 7.90 INR/hour 3. Throughput = 5.7474 requests/s x 3,600 = 20,690.6 requests/hour 4. Cost per 1,000 requests = 7.90 / 20,690.6 x 1,000 = **0.38 INR**

SLO. 99% of `/parse` requests return HTTP 200 within 30 s, measured over one standard load-test run (2 minutes). Error budget = 1% of 685 = **6.85, so 6 requests**. Used: **0** (0% of the budget).

Component	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	2,820 of 3,000 kept; labelling 26 min; 0 missing ids	single SeaweedFS Pod; one 282 MB single-request upload was dropped (Pod limit was 512 MiB)	multipart upload (now used) and replicated storage
Model quality	full frame 51% vs teacher 58%; 100% valid JSON	student copies teacher errors; tuned on validation	larger student, or logit distillation with a same-vocabulary teacher
Latency and throughput	p95 1,100 ms; 5.75 req/s; 0.23 s per command	one generation at a time per Pod on 1 CPU	request batching, int8 quantisation
Resource utilisation	~1,000m CPU and ~360 MiB per Pod	CPU-bound at its limit; memory limit 1 GiB is about 3x the use	lower the memory limit to 512Mi, raise CPU
Reliability and recovery	rollout 0/1,722 failed; recovery 13 s; cold start 9 s	single node, single laptop	multi-node cluster, PodDisruptionBudget
Security and cost	0 gitleaks findings; 0.38 INR per 1,000 requests	one shared S3 key with admin rights	per-service read-only keys; scale to 1 replica when idle